

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS:

Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I Ebin Antony P (192373011)_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1. Online Symposium Portal

1. Identify appropriate HTML5 semantic elements missing in the structure.

The given HTML form is missing several HTML5 semantic elements that improve the webpage structure.

Missing elements:

- <header> – Displays the page title.
- <main> – Contains the main content.
- <section> – Groups the registration form.
- <label> – Provides labels for input fields.
- <footer> – Displays footer information.

2. Modify the structure to make it standards-compliant.

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Symposium Portal</title>
</head>
<body>

<header>
  <h1>Online Symposium Registration</h1>
</header>

<main>
  <section>
    <form action="/register" method="POST">

      <label for="name">Name:</label>
      <input type="text" id="name" name="name" required>

      <br><br>

      <label for="email">Email:</label>
      <input type="email" id="email" name="email" required>

      <br><br>

      <button type="submit">Submit</button>

    </form>
  </section>
</main>

<footer>
```

<p>© 2026 Symposium Portal</p>
</footer>

</body>
</html>

3. Interpret the HTTP request generated during form submission.

When the student clicks the **Submit** button:

- The browser collects the values entered in the form.
- Since the method is **POST**, the data is sent in the request body.
- The request is sent to the server at the URL **/register**.

Example:

POST /register HTTP/1.1

Host: www.college.edu

Content-Type: application/x-www-form-urlencoded

[name=John&email=john@example.com](#)

Interpretation:

- **POST** – Sends form data to the server.
- **/register** – URL that processes the registration.
- **Host** – Website domain.
- **Content-Type** – Indicates form data format.
- **Request Body** – Contains the student's name and email.

4. Analyze the HTTP response cycle from server to browser.

The HTTP response cycle is:

1. Student fills out the registration form.
2. Browser sends a **POST** request to the server.
3. Server receives and validates the data.
4. Server stores the registration details in the database.
5. Server generates an HTTP response.
6. Browser receives the response.
7. Browser displays a success or error message to the user.

Example Response:

HTTP/1.1 200 OK

Content-Type: text/html

<html>

<body>

<h2>Registration Successful!</h2>

</body>

</html>

Common HTTP Status Codes:

- **200 OK** – Request successful.
- **400 Bad Request** – Invalid input.
- **404 Not Found** – Requested page not found.
- **500 Internal Server Error** – Server error.

5. *Suggest improvements for usability and accessibility.*

- Use semantic HTML5 elements such as <header>, <main>, <section>, and <footer>.
- Add <label> elements for all input fields.
- Use the required attribute for mandatory fields.
- Use proper input types such as email.
- Add placeholder text to guide users.
- Use type="submit" for the button.
- Display clear validation and error messages.
- Ensure keyboard accessibility.
- Maintain good color contrast for readability.
- Make the webpage responsive using the viewport meta tag.

Conclusion

The Online Symposium Portal should use semantic HTML5 elements, proper form validation, and the **HTTP POST** method for secure form submission. When the form is submitted, the browser sends an HTTP request to the server, which processes the data and returns an appropriate response. Improving accessibility, validation, and responsive design makes the portal more user-friendly and standards-compliant.

2: Cross-Browser Compatibility Issue

1. Identify improper nesting and missing closing tags.

The given HTML contains several errors:

- Missing `<!DOCTYPE html>` declaration.
- Missing lang attribute in the `<html>` tag.
- `<h1>` tag is not closed (`</h1>` missing).
- `<p>` tag is not closed (`</p>` missing).
- No semantic HTML5 elements are used.
- Improper HTML structure may lead to inconsistent rendering.

2. Analyze how different browsers may interpret this structure.

Different browsers have different HTML parsers. When HTML contains errors:

- Browsers automatically try to fix missing tags.
- Chrome may automatically close the `<h1>` before the `<p>`.
- Firefox and Edge may also correct the HTML, but the layout may differ slightly.
- Older browsers may display incorrect formatting or spacing.
- This results in inconsistent appearance across browsers.

3. Identify missing semantic elements (e.g., header, section).

The following HTML5 semantic elements should be used:

- `<header>` – Displays the website heading.
- `<main>` – Contains the main webpage content.
- `<section>` – Groups related content.
- `<footer>` – Displays footer information.

4. Provide a corrected W3C-compliant HTML structure.

These elements improve readability, accessibility, and SEO.

```
<html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Company</title>
```

```
</head>

<body>

<header>

  <h1>Welcome</h1>

</header>

<main>

  <section>

    <p>About us</p>

  </section>

</main>

<footer>

  <p>&copy; 2026 Company</p>

</footer>

</body>

</html>
```

5. Suggest techniques to ensure cross-browser compatibility.

To ensure consistent display across browsers:

- Use valid HTML5 and CSS according to W3C standards.
- Include the `<!DOCTYPE html>` declaration.
- Close all HTML tags properly.
- Use semantic HTML5 elements (header, main, section, footer).
- Test the website in multiple browsers (Chrome, Firefox, Edge, Safari).
- Use CSS resets or Normalize.css for consistent styling.
- Avoid browser-specific features unless necessary.
- Validate HTML and CSS using W3C validation tools.
- Make the webpage responsive using the viewport meta tag.
- Use modern, standards-compliant CSS and JavaScript.

Conclusion

The original HTML contains missing closing tags and lacks semantic structure, which can cause browsers to render the page differently. Using proper HTML5 syntax, semantic elements, and W3C standards ensures consistent layout, better accessibility, improved SEO, and reliable cross-browser compatibility.

3: Form Submission Failure

1. Identify issues in the form configuration.

The given form has the following issues:

- The button uses **type="button"** instead of **type="submit"**.
- Since it is a normal button, clicking it does not submit the form.
- The **GET** method is not suitable for sending sensitive data such as passwords.
- The password will appear in the URL when using GET.
- No validation (required) is provided for the input fields.

2. Analyse why the HTTP request is not triggered.

The HTTP request is not sent because:

- The button is defined as **type="button"**.
- A button of this type performs no action unless JavaScript is used.
- Since there is no JavaScript code to submit the form, clicking **Login** does not send any request to the server.

3. Interpret the role of GET method in this scenario.

The **GET** method:

- Sends form data as **URL query parameters**.
- Is mainly used to retrieve data from the server.
- Is **not secure** for login forms because the username and password become visible in the browser's address bar.
- Can be stored in browser history and server logs.

Example URL using GET:

/submit?username=John&password=12345

4. Purpose a corrected implementation

```
<html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Login Form</title>
```

```
</head>
```

```
<body>

<form action="/submit" method="POST">

  <label for="username">Username:</label>

  <input type="text" id="username" name="username" required>

  <br><br>

  <label for="password">Password:</label>

  <input type="password" id="password" name="password" required>

  <br><br>

  <button type="submit">Login</button>

</form>

</body>

</html>
```

5. Suggest improvements for secure data transmission.

To improve security:

- Use the **POST** method instead of **GET**.
- Use **HTTPS** to encrypt data during transmission.
- Add the required attribute for form validation.
- Use strong password validation rules.
- Store passwords securely using hashing on the server.
- Implement server-side validation in addition to client-side validation.
- Protect against CSRF attacks using CSRF tokens.
- Display clear error messages without revealing sensitive information.
- Use secure session management after successful login.

Conclusion

The form fails because the button uses **type="button"**, which does not submit the form. Additionally, using the **GET** method exposes sensitive information in the URL. Changing the button to **type="submit"**, using the **POST** method, and enabling **HTTPS** and proper validation ensures secure and successful form submission.

